

An **Operating System (OS)** is one of the most fundamental software components in a computer system.

It acts as a **bridge between the user and the hardware**, enabling efficient utilization of system resources such as CPU, memory, and I/O devices.

- A computer system is made up of **hardware** (CPU, memory, I/O devices) and **software** (programs).
- Users interact with computers via **applications** (games, browsers, editors).
- The **Operating System (OS)** is the **middle layer** that enables communication between **hardware and applications**.

Without an OS:

- Users would need to know **machine-level instructions**.
- Running programs and managing resources would be almost impossible.

Thus, the OS is essential for:

- **Abstraction** – Hides hardware complexity.
- **Convenience** – Makes computers user-friendly.
- **Efficiency** – Manages hardware resources optimally.

There are multiple perspectives to define an OS:

1. User View

- To a user, an OS is a **friendly interface** that allows them to run programs and perform tasks such as word processing, gaming, or browsing.

2. System View

- To the system, the OS is a **resource manager** that controls hardware resources like CPU scheduling, memory allocation, and device management.

3. Formal Definition

An **Operating System** is a system software that manages computer hardware and software resources and provides common services for computer programs.

Functions of an OS:

1. **Process Management** – Creating, scheduling, and terminating processes.
2. **Memory Management** – Allocation and deallocation of memory.
3. **File System Management** – Organizing and controlling access to data.

- 4. Device Management** – Managing I/O devices.
- 5. Security and Protection** – Ensuring authorized access.
- 6. User Interface** – CLI (Command Line Interface) or GUI (Graphical User Interface).

OS as a Resource Manager

- The OS allocates resources (CPU time, memory, I/O devices) among multiple programs.
- Example: If three programs want CPU, the OS schedules them.

OS as a Control Program

- It ensures that applications do not interfere with each other.
- Example: If a word processor tries to access memory used by a browser, OS blocks it.

OS as a Virtual Machine

- Provides an **abstract machine view** to users.
- Example: File systems make hard drives appear like hierarchical folders.

Design Goals of Operating System

When designing an OS, developers aim to achieve several objectives.

1. Primary Design Goals

(A) Convenience:

- The OS should make the computer system **easy to use**.
- Example: GUI systems (Windows, macOS) provide user-friendly environments.

(B) Efficiency:

- The OS should manage resources efficiently to maximize system performance.
- Example: CPU scheduling algorithms like Round Robin, Priority Scheduling.

(C) Reliability:

- The OS must be fault-tolerant and provide recovery mechanisms.
- Example: Checkpointing in UNIX systems.

(D) Scalability:

- The OS should support growth in workload or hardware.
- Example: Modern Linux kernels can scale from embedded devices to supercomputers.

2. Secondary Design Goals

1. Security

- Protect data and resources from unauthorized access.
- Example: User authentication, encryption.

2. Portability

- OS should be adaptable to different hardware platforms.
- Example: UNIX and Linux are portable across architectures.

3. Extensibility and Modularity

- The OS should allow easy updates or addition of features.
- Example: Microkernel architecture allows modular design.

4. Fairness

- The OS must allocate resources fairly among users and processes.

5. Performance

- Ensure minimum response time in interactive systems and maximum throughput in batch systems.

Goals Based on System Type

- **Batch Systems** → Focus on **throughput and CPU utilization**.
- **Interactive Systems** → Focus on **response time and user experience**.
- **Real-Time Systems** → Focus on **meeting deadlines and predictability**.
- **Distributed Systems** → Focus on **transparency and fault tolerance**.

Evolution of Operating Systems

Operating systems have evolved alongside computer hardware. We can divide this evolution into **generations/phases**:

1. First Generation (1940s–1950s) – No Operating System

- Early computers (ENIAC, EDVAC, UNIVAC) had **no OS**.
- Programs were entered using **plug boards, switches, and machine code**.
- Execution was manual and slow.

- Problems:
 - High setup time.
 - No resource management.
 - Only one program at a time.

2. Second Generation (1950s–1960s) – Batch Processing Systems

- Introduction of **punch cards** and **magnetic tape**.
- **Batch Processing**: Jobs grouped and executed one after another.
- No interaction between user and computer during execution.
- Example OS: **IBM 7094 batch systems**.

Advantages:

- Better utilization of CPU.

Disadvantages:

- No user interaction.
- Debugging was very slow.

3. Third Generation (1960s–1970s) – Multiprogramming and Time-Sharing

- **Multiprogramming**: Multiple jobs kept in memory, CPU switches between them.
- **Time-Sharing**: Each user gets small time slices → illusion of simultaneous execution.
- Development of **UNIX (1969)** at Bell Labs.
- Example OS: **IBM OS/360, Multics**.
- Features:
 - Interactive systems.
 - File systems introduced.
 - Better CPU utilization.

4. Fourth Generation (1970s–1990s) – Personal Computers and GUI

- Introduction of **microprocessors** → personal computers (PCs).
- Graphical User Interface (GUI) introduced.
- Example OS: **MS-DOS, Windows 95, Mac OS, UNIX variants**.
- Networking became popular → client-server model.

- Features:
 - Easy-to-use GUI.
 - Better security and portability.

5. Fifth Generation (1990s–Present) – Modern Operating Systems

- **Distributed and Networked OS** → Internet-driven.
- **Multiprocessor and Multicore support.**
- **Mobile OS:** Android, iOS.
- **Cloud OS:** Google Chrome OS.
- **Open-source OS:** Linux.
- Features:
 - Virtualization and containerization (Docker, Kubernetes).
 - Cloud computing and grid computing.
 - Real-time processing in embedded systems.

Generation	Key Features	Example OS
1st (1940s–50s)	No OS, manual execution	ENIAC, EDVAC
2nd (1950s–60s)	Batch processing	IBM 7094
3rd (1960s–70s)	Multiprogramming, Time-sharing	OS/360, UNIX
4th (1970s–90s)	GUI, Personal computing	MS-DOS, Mac OS
5th (1990s–Now)	Distributed, Mobile, Cloud	Linux, Windows, Android

Major Functions of Operating System

1. Process Management

- Create, schedule, and terminate processes.
- Example: Linux uses **Completely Fair Scheduler (CFS)**.

2. Memory Management

- Allocates and deallocates memory to processes.
- Techniques: Paging, Segmentation, Virtual Memory.

3. File Management

- Organizes data into files and directories.
- Example: NTFS (Windows), ext4 (Linux).

4. Device Management

- Manages device drivers and I/O devices.

5. Security and Protection

- Prevents unauthorized access (passwords, permissions, encryption).

6. Networking

- Provides protocols for distributed systems (TCP/IP stack).

7. User Interface

- CLI (Command Line Interface) → UNIX, Linux.
- GUI (Graphical User Interface) → Windows, macOS, Android.

Case Studies of OS

1. UNIX

- Developed in 1969 at Bell Labs.
- Portable, multi-user, multitasking.
- Inspired Linux and macOS.

2. Windows

- Developed by Microsoft.
- Evolved from **MS-DOS** → **Windows 95** → **Windows 10/11**.
- Known for GUI and widespread adoption.

3. Linux

- Open-source UNIX-like OS.
- Supports servers, desktops, mobiles, IoT.
- Kernel-based modular design.

4. Mobile OS

- Android: Linux-based, most widely used mobile OS.
- iOS: Apple's OS for iPhones/iPads, highly secure.

Structure of Operating System

An Operating System (OS) is a complex system software that manages hardware resources (CPU, memory, I/O devices) and provides services to applications and users.

Because of its complexity, the OS must be organized into well-defined structures.

- Structure = The internal design/organization of the OS.
- Different OS structures have evolved to balance performance, modularity, reliability, and security.

Why Do We Need OS Structures?

- Complexity: OS is one of the largest software systems (millions of lines of code).
- Maintainability: Without structure, debugging or extending OS is difficult.
- Modularity: Helps divide OS into manageable parts.
- Performance vs Flexibility: Structures try to balance efficiency with robustness.

Types of Operating System Structures

1. Monolithic Structure

- Entire OS is one large kernel running in supervisor mode.
- Every service (process, memory, I/O, file system) resides inside the same address space.

Examples

- MS-DOS
- UNIX v6, v7 (early versions)

Advantages

- Simple design.
- Efficient → No message passing overhead.
- Fast system call handling.

Disadvantages

- Poor maintainability.
- Insecure (all services share same memory).
- A single bug can crash entire OS.
- Difficult to extend.

2. Layered Structure

- OS is divided into **layers**, each built on top of the lower one.
- Layer n uses services of layer $(n-1)$.

Examples

- THE Operating System (Dijkstra, 1968).
- IBM System/360 OS.

Advantages

- Modular, easy to debug.
- Isolation → Higher layers protected from lower errors.
- Easier development.

Disadvantages

- Communication overhead between layers.
- Not flexible → strict layering may not suit all systems.

3. Microkernel Structure

- Only **essential functions** are inside kernel (IPC, CPU scheduling, memory mgmt).
- Other services (file system, device drivers, networking) run in **user mode** as servers.

Examples

- **Mach (Carnegie Mellon University).**
- **QNX, Minix.**
- Influence on Windows NT and macOS.

Advantages

- More secure (less code in kernel).
- Easier to extend/maintain.
- Fault isolation (bug in driver won't crash kernel).

Disadvantages

- Performance overhead due to message passing.
- More context switches between kernel ↔ user.

4. Modular Structure

- Kernel has a **core** + dynamically loadable modules.
- Modules provide additional services (drivers, filesystems).

Examples

- Linux kernel modules.
- Solaris modules.

Advantages

- Flexibility: load/unload modules dynamically.
- Efficient: close to monolithic performance.
- Good balance between simplicity & extensibility.

Disadvantages

- More complex design than monolithic.

5. Hybrid Structure

- Combines **monolithic performance** with **microkernel modularity**.
- Some services inside kernel, others outside.

Examples

- **Windows NT, Windows 10/11.**
- **macOS (XNU kernel).**

Advantages

- Good balance: performance + stability.
- Portable, supports multiple hardware platforms.

Disadvantages

- More complex design.

6. Virtual Machine Structure

- OS provides a **virtual copy of hardware** to each user.
- Each user can run their own OS on top of it.

Examples

- **IBM VM/370** (mainframe).
- **VMware, VirtualBox, Hyper-V.**

Advantages

- Strong isolation between users.
- Multiple OS on same machine.

Disadvantages

- Performance overhead.
- More hardware requirements.

7. Exokernel Structure

- Developed at MIT.
- Kernel provides **minimal resource protection**, leaving resource management to applications.

Example

- **Exokernel** research project.

Advantages

- Maximum flexibility.
- High performance (apps manage resources directly).

Disadvantages

- Complex application design.
- Less portability.

Functions of Operating System

- The Operating System (OS) is the manager of a computer system.
- It controls and coordinates the use of hardware among various applications and users.
- Without an OS, the user would have to manage CPU scheduling, memory allocation, I/O control manually.

Primary Functions of OS

1. Process Management

- A process = program in execution.
- OS manages creation, scheduling, and termination of processes.

Subfunctions

1. Process Creation & Deletion

- OS creates processes when a program starts.
- Example: In Linux → `fork()` system call.

2. CPU Scheduling

- Decides which process runs when multiple are ready.
- Algorithms: FCFS, SJF, Priority, Round Robin, Multilevel Queue.

3. Context Switching

- Saving and loading process state when switching CPU.

4. Inter-Process Communication (IPC)

- Allows processes to exchange data.
- Mechanisms: Pipes, Message Queues, Shared Memory.

5. Synchronization

- Coordinates processes to avoid race conditions.
- Tools: Semaphores, Monitors.

6. Deadlock Handling

- OS detects, prevents, or recovers from deadlocks.

2. Memory Management

- Memory = critical resource shared among processes.
- OS ensures fair and efficient allocation.

Subfunctions

1. Memory Allocation/Deallocation

- Assigns memory when process starts, reclaims when finished.

2. Address Translation

- Converts logical addresses (used by programs) to physical addresses (actual hardware).

3. Swapping

- Moves processes between main memory and disk when memory is full.

4. Paging & Segmentation

- Paging: Fixed-size blocks.
- Segmentation: Variable-sized logical units.

5. Virtual Memory

- Provides illusion of large memory using disk.
- Example: Windows “Pagefile.sys”.

3. File Management

- OS provides abstraction of files and directories to manage data.

Subfunctions

1. File creation, deletion, reading, writing.
2. Directory management (hierarchical structure).
3. Access control (permissions).
4. File sharing across users and processes.
5. File protection (against crashes or unauthorized access).

File System Examples

- FAT32 (Windows legacy), NTFS (Windows), ext4 (Linux), HFS+ (macOS).

4. Device Management

- OS manages input/output devices using device drivers.

Subfunctions

1. Device Drivers – Translate high-level commands into hardware-specific commands.
2. Buffering – Temporary storage of data during transfer.
3. Spooling – Overlapping I/O operations (e.g., print spooling).
4. Device Allocation – Assigns devices to processes fairly.

5. Security and Protection

- OS ensures system integrity and prevents unauthorized access.
 1. Authentication – Verifies identity (passwords, biometrics).
 2. Authorization – Grants access rights (read, write, execute).
 3. Encryption – Protects data in transit/storage.
 4. Access Control Lists (ACLs) – Define permissions for each file/device.

6. Networking and Communication

- In distributed and networked systems, OS provides networking functions.
- Example: TCP/IP stack built into Windows, Linux.
- Functions:
 - Packet routing, addressing, error handling.
 - Remote file access (NFS, SMB).
 - Remote login (SSH, Telnet).

7. User Interface

1. CLI (Command Line Interface)
 - Example: UNIX Shell, MS-DOS.
2. GUI (Graphical User Interface)
 - Example: Windows, macOS, Android.
3. Touch/Voice Interfaces
 - Example: Android/iOS, smart assistants.

Secondary Functions of OS

Resource Allocation

- OS acts as a resource manager.
- Ensures fairness among users/processes.
- Example: CPU quotas in Linux, scheduling in Windows.

Accounting

- OS keeps logs of resource usage.
- Useful for billing in cloud systems or detecting heavy users.

Error Detection & Recovery

- OS constantly monitors for errors.
- Examples:
 - Memory errors → parity/ECC handling.
 - Device errors → retries, timeouts.
 - Application crashes → error messages/logs.

System Performance Monitoring

- OS provides tools to monitor CPU, memory, disk, and network usage.
- Examples:
 - Windows Task Manager.
 - Linux top, htop.

Special Functions

Real-Time Operating System Functions

- Meet strict timing deadlines.
- Example: VxWorks, RTLinux.
- Functions:
 - Deterministic scheduling.
 - Priority inversion handling.
 - Minimal interrupt latency.

Distributed Operating Systems

- Manage resources across a network of computers.
- Functions:
 - Transparency (location, replication, concurrency).
 - Load balancing.
 - Fault tolerance.

Cloud and Mobile OS Functions

- Cloud OS (e.g., Chrome OS, Kubernetes OS-level management).
- Mobile OS (Android, iOS).
- Functions:
 - Energy management.
 - App sandboxing for security.
 - Cloud synchronization.

Process:

- A **process** is a **program in execution**.
- The Operating System (OS) is responsible for creating, scheduling, and terminating processes.
- Process management is one of the **most important functions** of the OS because the CPU is shared among many tasks.

Program vs Process

- **Program** = Passive entity (a file stored on disk).
- **Process** = Active entity (a program in execution, with resources assigned).

Process States:

A process does not always use the CPU; it changes states:

1. **New** → Process is being created.
2. **Ready** → Waiting to be assigned to the CPU.
3. **Running** → Currently executing on CPU.
4. **Waiting/Blocked** → Waiting for some I/O or event.
5. **Terminated** → Finished execution.

Process Control Block (PCB)

- A **process** is not just code; it requires **resources** such as CPU time, memory, files, and I/O devices.
- The **Operating System (OS)** must keep track of each process.
- To do this, it maintains a special data structure called the **Process Control Block (PCB)**.

A Process Control Block (PCB) is a **data structure maintained by the operating system** that stores all the information about a process.

- Each process in the system has its own PCB.
- When a process is created, a PCB is also created.
- When a process terminates, its PCB is destroyed.

Purpose of PCB

1. **Identification** – Uniquely identifies processes with Process ID (PID).
2. **Management** – Helps OS schedule processes, allocate memory, manage I/O.
3. **Protection** – Prevents one process from interfering with another.
4. **Context Switching** – Saves process state when switching between processes.
5. **Resource Tracking** – Keeps record of resources (files, devices) allocated to the process.

Structure of PCB

A PCB contains different fields, grouped into categories:

1. Process Identification Information

- **Process ID (PID):** Unique identifier for each process.
- **Parent Process ID (PPID):** The process that created this process.
- **User ID (UID):** Owner of the process (for security/accounting).

2. Process State Information

- Current state of the process:
 - New, Ready, Running, Waiting, Terminated.
- Helps OS know what the process is doing.

3. CPU Scheduling Information

- Process Priority.
- Scheduling Queue Pointers (ready queue, wait queue).
- Other scheduling parameters (e.g., CPU burst estimate for SJF scheduling).

4. CPU Context Information

- **Program Counter (PC):** Address of the next instruction.
- **Registers:** General-purpose registers, stack pointers, accumulator.
- **Flags:** Condition codes.

5. Memory Management Information

- Base Register (start of process memory).
- Limit Register (memory boundary).
- Page Table (for paging).
- Segment Table (for segmentation).

6. Accounting Information

- CPU usage (time used so far).
- Clock time (start/end time).
- Number of jobs completed.
- Job/billing information (in multi-user systems).

7. I/O Status Information

- List of I/O devices allocated.
- List of open files.
- Pending I/O requests.

Working of PCB in Context Switching

- When CPU switches from one process to another:
 1. Current process state (PC, registers, flags) is **saved in PCB**.
 2. The new process's PCB is loaded into CPU registers.
 3. Execution resumes from where the new process left off.

This ensures that processes run **independently without interfering** with each other.

Example:

- Process A running → interrupted by Process B.
- OS saves **A's context** in PCB(A).
- Loads **B's context** from PCB(B).
- Later resumes **A** from its saved state.

Contents of PCB:

1. **Process ID (PID)**
2. **Process State**
3. **Program Counter (PC)**
4. **CPU Registers**
5. **CPU Scheduling Info** (priority, queues)
6. **Memory Management Info** (base/limit, page tables)
7. **I/O Status Info**

Relationship Between PCB and Scheduling Queues

- All PCBs are linked together in different **queues**:
 - **Ready Queue** – List of PCBs waiting for CPU.
 - **I/O Queue** – List of PCBs waiting for I/O.

Importance of PCB

- **Efficient Scheduling** → Keeps track of priorities.
- **Multitasking Support** → Enables context switching.
- **Process Isolation** → Each process has its own PCB.
- **Security & Protection** → Tracks ownership (UID).
- **Resource Management** → Helps OS allocate/release resources properly.

Levels of Scheduling

1. **Long-term scheduling** → Decides which jobs are admitted to the system (job scheduling).
2. **Medium-term scheduling** → Suspends/resumes processes (for balancing).
3. **Short-term scheduling** → Decides which ready process gets CPU next (CPU scheduling).

CPU Scheduling Criteria

Performance is measured using:

1. **CPU Utilization** → Keep CPU as busy as possible.
2. **Throughput** → No. of processes completed per unit time.
3. **Turnaround Time** → Completion time – Arrival time.
4. **Waiting Time** → Time spent in ready queue.

5. Response Time → Time from request submission to first response.

6. Fairness → No process should starve indefinitely.

Types of Scheduling

- **Preemptive** → CPU can be taken away from a process.
- **Non-preemptive** → Once given, CPU is not taken until process finishes or blocks.

Scheduling Algorithms

1. First-Come, First-Served (FCFS)

- Non-preemptive.
- Processes executed in the order they arrive in the ready queue.
- Implemented using a FIFO queue.

Example

Processes: P1=24, P2=3, P3=3 (burst times)

Order: P1 → P2 → P3

2. Shortest Job Next (SJN) / Shortest Job First (SJF)

- Selects process with shortest CPU burst.
- Can be **non-preemptive** or **preemptive** (Shortest Remaining Time First – SRTF).

Example (Non-Preemptive)

Processes: P1=6, P2=8, P3=7, P4=3

Order: P4 (3) → P1 (6) → P3 (7) → P2 (8)

3. Priority Scheduling

- Each process has a **priority number**.
- CPU assigned to highest-priority process.
- Can be **preemptive** or **non-preemptive**.

Problem: **Starvation** – low-priority processes may never execute.

Solution: **Aging** – gradually increase priority of waiting processes

4. Round Robin (RR)

- Preemptive.
- Each process gets **fixed time quantum** (e.g., 10ms).
- If not finished, placed at end of queue.

Example:

Processes: P1=24, P2=3, P3=3, Quantum = 4

Execution:

P1(4), P2(3), P3(3), P1(20)

5. Multilevel Queue Scheduling

- Ready queue divided into **separate queues**:
 - Foreground (interactive).
 - Background (batch).
- Each queue has its own scheduling algorithm.
- CPU allocated between queues using fixed priority or time slicing.

Problem: Rigid → processes permanently assigned to queues.

6. Multilevel Feedback Queue Scheduling (MLFQ)

- Improvement over Multilevel Queue.
- Processes can move between queues based on execution history.
- Example:
 - New process starts in high-priority queue.
 - If it consumes too much CPU, moved to lower queue.
- Used in modern systems (e.g., UNIX, Linux).

Context Switching

- When CPU switches from one process to another, OS must save the state of the old process (PCB) and load the state of the new process.
- Overhead (since no useful work is done).
- Faster context switch = better performance.

Inter-Process Communication (IPC)

Processes need to **communicate and synchronize** with each other.

Models of IPC

1. Shared Memory

- Processes share a region of memory.
- Faster but requires synchronization.

2. Message Passing

- Processes send/receive messages via OS.
- Safer but slower.

Memory Management

Memory Management is the process of controlling and coordinating computer memory, assigning memory blocks to various running processes to optimize overall system performance.

Memory = The **storage space directly accessible by CPU**.

- CPU executes instructions that must reside in main memory (RAM).
- Since main memory is limited and expensive, OS must manage memory efficiently.

Objectives of Memory Management

1. Efficient Utilization of Memory

- Memory is limited, so it must be used effectively without wastage.
- OS allocates memory blocks to processes in such a way that maximum processes can run simultaneously.
- Example: Paging avoids external fragmentation and ensures efficient usage.

2. Multiprogramming Support

- Multiple processes must coexist in memory.
- OS divides memory dynamically and allows many processes to share it.
- Improves CPU utilization (while one process waits for I/O, another can run).

3. Process Protection and Isolation

- One process should not interfere with another process's memory.
- Example: Process A should not overwrite Process B's data.
- OS provides protection using base and limit registers or MMU hardware.
- Also protects kernel memory from user processes.

4. Relocation and Transparency

- Programs may be moved in memory (e.g., due to swapping, compaction).
- Processes should not be aware of their physical memory addresses.
- OS provides logical addresses (virtual addresses) to achieve location transparency.

5. Abstraction and Virtual Memory

- OS provides an illusion of large memory using disk space.
- Virtual memory allows execution of large programs even if RAM is small.
- Simplifies programming, since developers need not worry about physical memory limits.

6. Fast Allocation and Deallocation

- When a process is created, OS must quickly assign memory.
- When it terminates, memory must be freed for reuse.
- Efficient allocation reduces overhead and improves system throughput.

7. Support for Sharing and Communication

- Some processes may need to share memory for communication.
- Example: Producer–Consumer problem (using shared buffer).
- OS provides controlled sharing without violating protection.

8. Performance Optimization

- Reduce the number of page faults.
- Use efficient page replacement algorithms (LRU, OPT, FIFO).
- Maintain balance between CPU utilization and I/O wait time.

9. Support for Swapping

- OS may temporarily swap out inactive processes to disk.
- Helps in increasing multiprogramming degree.
- Memory management ensures smooth swap in / swap out operations.

10. Security and Reliability

- Protect sensitive memory areas from unauthorized access.
- Prevents malware or faulty processes from damaging system memory.
- Example: Kernel memory protection in Linux/Windows.

Memory Hierarchy:

Memory Hierarchy refers to the arrangement of different storage devices in a computer system in a pyramid-like structure, based on their **speed, cost, and capacity**.

- Faster memory = smaller & expensive.
- Slower memory = larger & cheaper.

Characteristics of Memory

Before understanding hierarchy, let's define key parameters:

1. **Access Time (Latency):** Time to read/write data.
2. **Memory Capacity:** Amount of data stored.
3. **Cost per Bit:** Price of storing data per bit.
4. **Volatility:** Whether data is lost when power is turned off.

Memory Hierarchy

- **Registers** (fastest, smallest, managed by CPU).
- **Cache Memory** (fast but expensive).
- **Main Memory (RAM)** → managed by OS.
- **Secondary Storage (Disk/SSD)** → used for **virtual memory**.

1. CPU Registers

- Located inside the **processor**.
- Extremely fast (access time = 1 cycle).
- Store temporary values: operands, addresses, intermediate results.
- Size is very small (few KB).
- Volatile → contents lost when power is off.

Examples:

- Program Counter (PC)
- Instruction Register (IR)
- General-purpose registers (AX, BX in x86).

2. Cache Memory:

- Small, high-speed memory between CPU and RAM.
- Stores frequently accessed instructions/data.
- Divided into **L1, L2, L3** caches.
 - **L1**: Fastest, smallest (32KB–512KB).
 - **L2**: Larger but slower (256KB–8MB).
 - **L3**: Shared among cores, largest (2MB–64MB).

Principle of Locality justifies cache:

- **Temporal Locality** → recently used data will likely be used again.
- **Spatial Locality** → nearby data is likely to be accessed soon.

3. Main Memory (RAM)

- Directly accessible by CPU.
- Volatile (data lost on shutdown).
- Larger capacity (GBs).
- Slower than cache, faster than disk.
- Types:
 - **DRAM (Dynamic RAM)**: Cheap, high density (used in main memory).
 - **SRAM (Static RAM)**: Faster but expensive (used in cache).

4. Secondary Storage

- Non-volatile (data persists without power).
- Large capacity (TBs).
- Much slower than RAM.
- Examples: HDDs, SSDs, Flash memory.

SSDs (Solid State Drives) are faster than HDDs, since they use flash storage.

5. Tertiary Storage

- Used for **backup and archival**.
- Very large capacity, very slow.
- Examples: Magnetic tapes, Optical disks (CD/DVD/Blu-ray), Cloud storage.

Memory Allocation Techniques:

Contiguous Memory Allocation

- Each process occupies **one contiguous block** in memory.

1. Fixed Partitioning

- Memory divided into **fixed-size partitions** at system startup.
- One process per partition.

Advantages:

- Simple to implement.
- Easy to allocate/free memory.

Disadvantages:

- **Internal Fragmentation** → unused space inside partitions.
- **Limited Multiprogramming** → number of partitions = number of processes.

2. Dynamic Partitioning

- Memory divided into **variable-sized partitions**, created at runtime.

Allocation Strategies:

- **First Fit** → Allocate first hole that is large enough.
- **Best Fit** → Allocate smallest hole that fits.
- **Worst Fit** → Allocate largest hole.

Problem:

- **External Fragmentation** → free memory scattered across locations.

Non-Contiguous Allocation

1. Paging

- Memory divided into **fixed-size blocks**:
 - Logical memory → **Pages**.
 - Physical memory → **Frames**.
- OS uses **Page Table** to map pages → frames.

Advantages:

- No external fragmentation.
- Supports virtual memory.

Disadvantages:

- **Internal Fragmentation** (last frame not fully used).
- Overhead of page table.

2. Segmentation

- Memory divided based on **logical modules** (functions, arrays, stack).
- Each segment has: **Base Address + Limit Size**.

Advantages:

- Matches programmer's view of memory.
- Different segments can grow independently.

Disadvantages:

- Causes **external fragmentation**.

3. Segmented Paging (Hybrid)

- Logical address → (Segment No., Page No., Offset).
- Combines **programmer-friendly segmentation + efficient paging**.

Virtual Memory:

Virtual Memory is a **memory management technique** that gives the illusion of a very large main memory by using **secondary storage (disk/SSD)**.

Benefits:

1. Run programs larger than physical memory.
2. Multiprogramming.
3. Simplified memory management.

7. Demand Paging

- Pages are loaded into memory **only when required**.
- If the page is not in memory → **Page Fault** occurs.

Steps in Page Fault Handling:

1. OS detects missing page.
2. Brings page from disk into memory.
3. Updates page table.
4. Restarts process from faulting instruction.

Page Replacement Algorithms:

In **Virtual Memory Systems**, processes may require more pages than the available physical frames.

When a page fault occurs (requested page is not in memory), the OS must decide:

1. Which page to remove from memory?
2. Which new page to load into memory?

Objectives of Page Replacement Algorithms

1. **Minimize Page Faults** → reduce delays.
2. **Efficient Memory Utilization**.
3. **Fairness** → all processes should get a fair share of memory.
4. **Low Overhead** → algorithm must be simple to implement.

Types of Page Replacement Algorithms:

1. FIFO (First-In-First-Out)

- The oldest loaded page is replaced first.
- Implemented using a **queue**.

Example:

Reference string = 7, 0, 1, 2, 0, 3, 0, 4, 2, 3, 0, 3, 2

Frames = 3

FIFO Page Replacement Table (you can draw in notes):

Page faults: 9

Advantages:

- Simple, easy to implement.

Disadvantages:

- Suffers from **Belady's Anomaly** (increasing frames may increase page faults).

2. Optimal (OPT / MIN)

- Replace the page that will not be used for the **longest time in future**.
- Requires **future knowledge**, so only theoretical.

Advantages:

- Produces minimum page faults.
- Used as a **benchmark**.

Disadvantages:

- Not practical (future references unknown).

3. LRU (Least Recently Used):

- Replace the page that has **not been used for the longest time**.
- Based on **temporal locality**.

Implementation Methods:

1. **Counter Method** → each page stores last access time.
2. **Stack Method** → maintain stack of recently used pages.

Example:

Same reference string, Frames = 3 → page faults: 7

Advantages:

- Approximates OPT well.
- No Belady's Anomaly.

Disadvantages:

- Requires hardware support (counters or stack).

4. LFU (Least Frequently Used)

- Replace the page with **lowest access frequency**.
- Idea: rarely used pages are less likely to be used again.

Advantages:

- Good for programs with repetitive patterns.

Disadvantages:

- May keep an old page forever if it was heavily used earlier.

5. MFU (Most Frequently Used)

- Replace the page with the **highest frequency count**.
- Assumption: A page used often has already done its job and may not be needed soon.

Belady's Anomaly

It is a phenomenon in some page replacement algorithms (notably **FIFO**) where **increasing the number of page frames results in an increase in the number of page faults** for a given reference string.

Why Does Belady's Anomaly Occur?

- FIFO replaces pages **blindly based on arrival order**, not based on actual future use.
- When frames increase, FIFO may replace **important pages** too early.
- Thus, instead of reducing page faults, the faults may increase.

Algorithms Affected

- **Affected:** FIFO, Random Replacement.
- **Not Affected:** LRU, LFU, Optimal.
 - These are **stack algorithms**, meaning the set of pages in memory with n frames is always a subset of the set with $(n+1)$ frames.
 - Thus, page faults cannot increase.

Paging in Operating Systems:

Paging is a memory management technique used by operating systems to eliminate the problem of external fragmentation.

In paging, both logical memory (process address space) and physical memory are divided into fixed-size blocks:

- Logical blocks $\rightarrow$ Pages
- Physical blocks $\rightarrow$ Frames

- **Page:** Fixed-size block of the logical address space.
- **Frame (or Page Frame):** Fixed-size block of physical memory.
- **Page Size = Frame Size** (so mapping is easy).
- **Page Table:** A data structure maintained by OS that keeps track of which page is stored in which frame.

Address Translation in Paging

Each **logical address** generated by CPU is divided into two parts:

1. **Page Number (p):** Index into the page table.
2. **Page Offset (d):** Displacement within the page.

Advantages of Paging

1. Eliminates **external fragmentation**.
2. Allows **efficient use of memory**.
3. Supports **virtual memory** (with demand paging).
4. Simplifies memory allocation (just need free frame list).
5. Allows processes to use more memory than physically available.

Disadvantages of Paging

1. **Internal fragmentation** (last page may not be completely used).
2. **Overhead of page tables** (can be very large for big processes).
3. Address translation requires **extra memory access** → slower.
 - Solution: **TLB (Translation Lookaside Buffer)**.
4. Complex memory management.

Translation Lookaside Buffer (TLB)

- A special **cache** used to speed up paging.
- Stores recent **page table entries**.
- If page number is in TLB → **fast translation** (TLB hit).
- If not → access page table in memory (TLB miss).

Paging in Virtual Memory

- **Demand Paging** → Pages are loaded only when required.
- OS keeps track of:
 - Pages in memory.
 - Pages on disk (swap space).
- If a required page is not in memory → **Page Fault** occurs → OS loads page from disk.

Segmentation in Operating Systems:

Segmentation is a memory management technique in which the logical address space of a process is divided into variable-sized segments.

Each segment represents a logical unit of a program such as:

- Main program
- Function
- Array
- Stack
- Data structure

Characteristics of Segmentation

1. Segments are **variable-sized**.
2. Segments correspond to **logical program units**.
3. Provides **protection & sharing** at segment level.
4. Eliminates **internal fragmentation**, but causes **external fragmentation**.

Advantages of Segmentation

1. **Logical View of Memory** → closer to programmer's view.
2. **No Internal Fragmentation** (since segments are variable).
3. **Supports Sharing & Protection:**
 - Example: A shared library can be mapped to multiple processes as a segment.
4. **Dynamic Growth:** A stack segment can grow/shrink dynamically.
5. **Supports Modular Programming** (each module = segment).

Disadvantages of Segmentation

1. **External Fragmentation** → free spaces may be scattered.
2. **Compaction Needed** → to overcome fragmentation, expensive compaction is required.
3. **Complex Memory Allocation** compared to paging.
4. **Variable Segment Sizes** → harder to manage than fixed-size pages.

Concurrent Processes:

A modern OS allows multiple processes to execute simultaneously. When two or more processes are executed overlapping in time (either truly parallel on multiple CPUs or interleaved on a single CPU), they are called Concurrent Processes.

Reasons for Concurrency

1. **Resource Utilization** → CPU is not idle while one process waits for I/O.
2. **Improved Throughput** → More work is completed in the same time.
3. **Fairness** → All users get CPU time.
4. **Responsiveness** → Interactive systems remain responsive even when multiple tasks are running.

Characteristics of Concurrent Processes

- **Independence** → Processes may be independent or cooperating.
- **Interleaving** → Execution steps are interleaved by scheduler.
- **Communication** → Cooperating processes may need to exchange data.
- **Synchronization** → Processes may need to coordinate their actions.

Types of Concurrent Processes

1. Independent Processes

- Do not share data with other processes.
- Output of one does not affect the other.
- Example: A compiler and a music player running at the same time.

2. Cooperating Processes

- Share data/resources and may affect each other.
- Example: Producer-Consumer (one process produces data, the other consumes it).

Problems in Concurrency:

Concurrency introduces several **challenges**:

1. Race Conditions

- A **race condition** occurs when multiple processes access and manipulate shared data concurrently, and the outcome depends on the order of execution.
- Example: Two processes withdrawing money from the same bank account at the same time → incorrect balance if not synchronized.

2. Critical Section Problem

- A **critical section** is a code segment where a process accesses shared data.
- Only one process at a time should execute in the critical section.
- Requirements of a correct solution:
 1. **Mutual Exclusion** → Only one process can enter critical section at a time.
 2. **Progress** → If no one is in critical section, processes should not be blocked.
 3. **Bounded Waiting** → A process must not wait forever.

3. Deadlock

- A situation where two or more processes wait indefinitely for resources held by each other.
- Example: Process A waits for resource held by Process B, and B waits for A.

4. Starvation

- A process waits indefinitely because higher-priority processes keep getting resources.

5. Data Inconsistency

- Without proper synchronization, shared data may become inconsistent.

Advantages of Concurrent Processes:

1. Better **CPU utilization**.
2. Faster **response time**.
3. Increased **throughput**.
4. Supports **multiprogramming and multitasking**.
5. Improves **system performance**.

Disadvantages / Challenges:

1. **Complexity** of design and debugging.
2. May cause **deadlock, race conditions, and starvation**.
3. Requires additional **synchronization mechanisms**.
4. **Context switching overhead** reduces efficiency.

Deadlock:

- In multiprogramming systems, multiple processes compete for limited resources (CPU, I/O devices, memory, files, printers, etc.).
- Sometimes, processes hold resources while waiting for other resources.
- If a set of processes is waiting indefinitely for resources that are held by each other, the system is said to be in a deadlock state.

System Model for Deadlock:

- Resources are classified into types: CPU cycles, I/O devices, files, semaphores, etc.
- Each resource type has instances (e.g., 3 printers, 2 CD drives).
- A process can:
 - Request a resource
 - Use the resource
 - Release the resource

Deadlock occurs when requests cannot be satisfied because resources are held by waiting processes.

Necessary Conditions for Deadlock:

- 1. Mutual Exclusion:** At least one resource is non-shareable (only one process can use it at a time).
- 2. Hold and Wait:** A process holding at least one resource is waiting for additional resources.
- 3. No Preemption:** A resource cannot be forcibly taken away from a process.
- 4. Circular Wait:** A circular chain of processes exists, each waiting for a resource held by the next process.

Methods for Handling Deadlock

1. Deadlock Prevention: Ensure that one of the Coffman conditions never holds.

- Eliminate **mutual exclusion** (make resources sharable wherever possible).
- Eliminate **hold and wait** (processes must request all resources at once).
- Eliminate **no preemption** (allow resources to be taken away).
- Eliminate **circular wait** (impose ordering of resources).

2. Deadlock Avoidance: Dynamically check resource allocation to ensure deadlock never occurs.

- Most famous method → **Banker's Algorithm**.
- System ensures that it never enters an **unsafe state**.

3. Deadlock Detection: Allow deadlock to occur, then detect it using algorithms.

- Use **Wait-for Graphs** (for single instance resources).
- Use **Detection Algorithms** (for multiple instances).

4. Deadlock Recovery: After detection, recover by:

- **Process Termination** → Kill one or more processes to break the cycle.
- **Resource Preemption** → Take resources away from processes.

Deadlock Prevention

1. Mutual Exclusion

- Cannot be entirely eliminated (e.g., printers cannot be shared).
- But some resources (like read-only files) can be made sharable.

2. Hold and Wait

- Require processes to request all resources at once.
- Problem: leads to low resource utilization & starvation.

3. No Preemption:

- If a process holding resources requests another that is not available, release all held resources.
- Practical for CPU & memory but not for I/O devices.

4. Circular Wait

- Impose a global ordering of resources (e.g., $R1 < R2 < R3$).
- Processes must request resources in increasing order.

Deadlock Avoidance – Banker's Algorithm:

- Designed by **Edsger Dijkstra**.
- Works like a banker lending money → ensures that resources are allocated only if the system remains in a **safe state**.

Steps:

1. Each process declares the **maximum resources** it may need.
2. System checks if resources can be allocated safely.
3. If yes → allocate; if no → process waits.

Advantage: Prevents deadlock.

Disadvantage: Requires advance information; complex for large systems.

Deadlock Detection:

- Allow system to enter deadlock and then **detect**.

For Single Instance Resources

- Use **Wait-for Graphs**.
- If cycle exists → deadlock.

For Multiple Instance Resources

- Use detection algorithms similar to Banker's Algorithm.

Recovery from Deadlock:

1. Process Termination

- Kill all deadlocked processes (costly).
- Kill one process at a time until deadlock breaks.

2. Resource Preemption

- Take resources away and give to others.
- Issues: selection of victim, rollback, starvation.

Advantages & Disadvantages of Deadlock Handling:

Advantages

- Ensures system stability.
- Improves resource utilization (with avoidance).
- Provides fairness to all processes.

Disadvantages:

- Prevention reduces concurrency.
- Avoidance needs prior knowledge.
- Detection & recovery can be expensive.

Semaphore:

- In concurrent processing, multiple processes may need to access shared resources.
- To prevent race conditions and maintain data consistency, processes must be synchronized.
- Semaphore is one of the earliest and most powerful synchronization tools introduced by Edsger Dijkstra in 1965.

Characteristics of Semaphores:

1. Integer variable (≥ 0).
2. Accessed only via **two atomic operations**: wait() and signal().
3. Used for **mutual exclusion** (only one process in critical section) or **resource counting** (track availability of resources).
4. Ensures that **race conditions** are avoided.

Operations on Semaphore:

i) wait() (also called P operation / down)

```
wait(S) {  
    while (S <= 0) ; // busy waiting  
    S = S - 1;  
}
```

- Decreases the value of semaphore S.
- If $S > 0 \rightarrow$ process continues.
- If $S = 0 \rightarrow$ process waits (blocked).

ii) signal() (also called V operation / up)

```
signal(S) {  
    S = S + 1;  
}
```

- Increases the value of semaphore S.
- If other processes are waiting, one gets unblocked.

Types of Semaphores

1. Binary Semaphore

- Value of semaphore is **0 or 1**.
- Works like a lock → ensures **mutual exclusion**.
- Example: Only one process can access a printer at a time.

2. Counting Semaphore

- Value can range over non-negative integers.
- Used to control access to a resource with **multiple instances**.
- Example: If 3 identical printers are available, semaphore initialized to 3.

Synchronization Problems using Semaphores:

1. Producer–Consumer Problem

- Producers generate data and put it into a buffer.
- Consumers take data out of the buffer.
- Buffer has limited size.
- Use semaphores to avoid overflow/underflow.

2. Readers–Writers Problem

- Multiple readers can read simultaneously.
- Only one writer can write (no readers during write).
- Semaphores coordinate between readers and writers.

3. Dining Philosophers Problem

- Five philosophers alternately think and eat.
- Each philosopher requires two forks (resources).
- Semaphores used to avoid deadlock and starvation.

Advantages of Semaphores

1. Simple and powerful synchronization tool.
2. Works for both **single instance and multiple instance resources**.
3. Can be implemented in both **kernel-level** and **user-level** programs.
4. Prevents **race conditions**.

Disadvantages of Semaphores

1. **Busy Waiting:** In simple implementation, processes keep checking semaphore (CPU waste).
2. **Deadlock:** Improper semaphore usage may cause deadlock.
3. **Priority Inversion:** Low-priority process holding semaphore may delay high-priority process.
4. **Complexity:** Misuse of wait/signal order leads to errors.

File Management

- A file is a collection of related information (data, instructions, or records) that is stored on secondary storage (like a hard disk, SSD, or cloud).
- File management is the set of OS functions that manage the storage, retrieval, naming, sharing, and protection of files.

Example:

- In Windows → Documents, Images, Programs (stored as .docx, .jpg, .exe)
- In Linux/Unix → Everything is treated as a **file** (including devices).

Objectives of File Management:

1. **Efficient Storage** – Use disk space optimally.
2. **Data Sharing** – Allow multiple processes/users to access files.
3. **Data Protection** – Protect from unauthorized access.
4. **Data Reliability** – Ensure no data corruption or loss.
5. **Simplification for User** – Provide abstraction (users don't need to worry about how/where data is stored).

Functions of File Management

1. File Creation and Deletion

- OS allows users/programs to **create new files** and store data in them.
- Provides system calls like create(), open(), close(), delete().
- Handles metadata (name, type, size, date, etc.).
- **Example:** touch myfile.txt in Linux creates a file.

2. File Manipulation (Read, Write, Update, Append)

- Files are not useful unless data can be accessed or modified.
- OS provides operations:
 - **Read** → Load data from file.
 - **Write** → Add or modify data.
 - **Update** → Change existing content.
 - **Append** → Add data at the end.
- **Example:** Opening a .docx file in MS Word to edit.

3. File Organization

- The OS decides how data is structured internally:
 - **Sequential** → Access one record after another.
 - **Direct (Random)** → Jump to any location.
 - **Indexed** → Use index tables for faster retrieval.
- Ensures **efficient storage and access**.
- **Example:** Database files use indexed access for queries.

4. File Directory Management

- Directories are used to **group files for easier navigation**.
- OS provides structures like:
 - Single-level, two-level, tree-structured, acyclic graph.
- Allows operations: list, search, rename, traverse.
- **Example:** Windows Explorer shows hierarchical directories.

5. File Mapping and Disk Management

- OS translates **logical file names into physical disk addresses**.
- Handles **allocation methods**:
 - Contiguous, Linked, Indexed allocation.
- Keeps track of **free space** on disk.
- **Example**: NTFS uses Master File Table (MFT) for mapping.

6. File Sharing and Concurrency Control

- Files may be shared among multiple processes/users.
- File management ensures:
 - Controlled access to avoid conflicts.
 - Locking mechanisms to prevent corruption.
- **Example**: Google Docs allows multiple users to edit the same file.

7. File Protection and Security

- Protects files from **unauthorized access or modification**.
- Methods:
 - Access Control Lists (ACLs) → Permissions (Read, Write, Execute).
 - Passwords.
 - Encryption.
- **Example**: In Linux → file permissions `rwxr-xr--`.

8. File Backup and Recovery

- OS maintains copies of files to prevent **data loss** due to crash, corruption, or accidental deletion.
- Provides **restore mechanisms**.
- **Example**: Windows "File History" or Linux `rsync`.

9. File Metadata Management

- OS stores and manages **file attributes** such as:
 - File name
 - File type (text, binary, image, executable)
 - Size
 - Owner and permissions
 - Time stamps (created, modified, accessed)
- This metadata is stored in **File Control Blocks (FCB)** or inodes (in Unix/Linux).

10. File Access Control

- OS defines how files can be accessed:
 - **Sequential Access** → Used in tapes, logs.
 - **Direct Access** → Used in databases.
 - **Indexed Access** → Used in large datasets.
- Prevents illegal or unsafe access.

11. File Abstraction for User

- Provides a **logical view of files** independent of hardware details.
- Hides complexity of physical storage.

File Attributes

Every file has **metadata** (stored in File Control Block):

- File name
- File type (text, binary, image, executable)
- File size
- Creation/modification date
- Owner and permissions
- Location on disk

File Operations

OS provides **system calls** for file operations:

1. **Create** – Create an empty file.
2. **Open** – Load file metadata into memory.
3. **Read** – Retrieve data from file.
4. **Write** – Modify/add data.
5. **Seek** – Move file pointer to a position.
6. **Close** – Release file resources.
7. **Delete** – Remove file permanently.

File Structure

- **Unstructured (Stream of Bytes)** → No specific structure, OS doesn't care (e.g., Unix text file).
- **Record-based** → Data stored as fixed-length records (used in databases).
- **Complex structures** → Nested objects, multimedia files, etc.

Advantages of File Management

1. Easy data organization.
2. Provides data sharing.
3. Ensures data security.
4. Improves system performance.
5. Reliable storage & recovery.

Disadvantages / Challenges

1. Overhead of maintaining metadata.
2. Fragmentation reduces performance.
3. Complexity in handling concurrent access.
4. Security risks (unauthorized access, malware).

Secondary Storage Management:

- Primary storage (RAM) is volatile and limited in size.
- To handle large amounts of data permanently, computers use secondary storage devices like Hard Disk Drives (HDDs), Solid-State Drives (SSDs), Magnetic Tapes, Optical Disks, Flash Drives, etc.
- The Operating System (OS) is responsible for managing these secondary storage devices efficiently.

Components:

- Disk Structure – Physical and logical organization of storage.
- Disk Scheduling – Algorithms to decide order of servicing I/O requests.
- Disk Formatting & Boot Block – Preparing storage for use.
- Disk Partitioning – Dividing disk into logical sections.
- Free-Space Management – Keeping track of available blocks.
- Buffering & Caching – Improving performance.
- RAID (Redundant Array of Independent Disks) – Reliability and fault tolerance

Disk Structure

- A magnetic disk has:
 - Platters (circular disks coated with magnetic material).
 - Tracks (concentric circles).
 - Sectors (subdivisions of a track).
 - Cylinders (set of tracks at the same radius across platters).
- Disk address = (cylinder number, track number, sector number).

Access time consists of:

1. Seek Time – Time to move head to correct track.
2. Rotational Latency – Time waiting for correct sector to rotate under head.
3. Transfer Time – Time to actually read/write data.

Disk Scheduling Algorithms:

1. First Come First Serve (FCFS)

- Requests are processed in the order they arrive.

Example:

- Disk queue: [98, 183, 37, 122, 14, 124, 65, 67]
- Head starts at 53.
- Movement: $53 \rightarrow 98 \rightarrow 183 \rightarrow 37 \rightarrow 122 \rightarrow 14 \rightarrow 124 \rightarrow 65 \rightarrow 67$
- Total head movement = 640 cylinders.

2. Shortest Seek Time First (SSTF)

- Selects the request closest to current head position.

Example:

- Queue: [98, 183, 37, 122, 14, 124, 65, 67], head = 53
- Closest to 53 = 65 → then 67 → 37 → 14 → 98 → 122 → 124 → 183
- Total head movement = 236 cylinders.

3. SCAN (Elevator Algorithm)

- Disk head moves like an elevator:
 - Moves in one direction, servicing requests.
 - When it reaches the end, reverses direction.

Example:

- Head = 53, direction → higher cylinders
- Queue: [98, 183, 37, 122, 14, 124, 65, 67], max = 199
- Movement: 53 → 65 → 67 → 98 → 122 → 124 → 183 → 199 → back → 37 → 14
- Total head movement = 382 cylinders.

4. C-SCAN (Circular SCAN)

- Head moves in one direction only.
- When end is reached, it jumps back to start without servicing requests during return.

Example:

- Head = 53, direction → higher cylinders
- Queue: [98, 183, 37, 122, 14, 124, 65, 67]
- Movement: 53 → 65 → 67 → 98 → 122 → 124 → 183 → 199 → jump → 0 → 14 → 37
- Total head movement = 382 cylinders.

5. LOOK Algorithm

- Similar to SCAN, but head stops at last request in direction instead of going to end of disk.

Example:

- Head = 53, direction $\rightarrow$ higher
- Queue: [98, 183, 37, 122, 14, 124, 65, 67]
- Movement: $53 \rightarrow 65 \rightarrow 67 \rightarrow 98 \rightarrow 122 \rightarrow 124 \rightarrow 183 \rightarrow \text{reverse} \rightarrow 37 \rightarrow 14$
- Total movement = 299 cylinders.

6. C-LOOK Algorithm

- Like C-SCAN but head only goes up to last request, then jumps to lowest request.

Example:

- Head = 53
- Movement: $53 \rightarrow 65 \rightarrow 67 \rightarrow 98 \rightarrow 122 \rightarrow 124 \rightarrow 183 \rightarrow \text{jump} \rightarrow 14 \rightarrow 37$
- Total movement = 322 cylinders.

UNIX is a powerful, multiuser, multitasking operating system originally developed at **AT&T Bell Labs (1969–70 by Ken Thompson & Dennis Ritchie)**.

Key features:

- Portability (written in C).
- Multitasking & multiuser.
- Hierarchical file system.
- Powerful shell & scripting support.

UNIX Architecture

1. **Kernel** – Core of OS, manages CPU, memory, files, devices.
2. **Shell** – Command interpreter, interface between user & kernel.
3. **File System** – Hierarchical structure (everything is a file).
4. **Utilities/Commands** – Programs for file manipulation, process management, text processing.

UNIX File System

- Organized in a **tree structure** starting from root /.
- Types of files:
 - Ordinary files (text, binary).
 - Directory files.
 - Special files (device files).
- Commands: ls, pwd, cd, cat, cp, mv, rm.

Shell in UNIX

- A **shell** is a command-line interpreter.
- Popular shells: **sh, bash, ksh, csh, zsh**.
- Functions:
 - Execute commands.
 - Provide scripting language for automation.
 - Handle input/output redirection, pipelines.

UNIX and Linux Commands

File and Directory Management Commands

1. **pwd** – Displays the current working directory.
 - Example: `pwd` → `/home/user`
2. **ls** – Lists files and directories in the current location.
 - Options: `ls -l` (long listing), `ls -a` (show hidden files).
3. **cd** – Changes the current working directory.
 - Example: `cd /etc`
4. **mkdir** – Creates a new directory.
 - Example: `mkdir project`
5. **rmdir** – Removes an empty directory.
 - Example: `rmdir old_dir`
6. **rm** – Deletes files or directories.
 - Example: `rm file.txt`, `rm -r folder/`
7. **cp** – Copies files or directories.
 - Example: `cp a.txt b.txt`, `cp -r dir1 dir2`

8. `mv` – Moves or renames files.

- Example: `mv old.txt new.txt`

9. `touch` – Creates an empty file or updates file timestamp.

- Example: `touch notes.txt`

10. `cat` – Displays the contents of a file.

- Example: `cat file.txt`

11. `more` – Shows file contents one page at a time.

12. `less` – Similar to `more` but with better navigation.

13. `head` – Shows the first 10 lines of a file.

- Example: `head -n 5 file.txt`

14. `tail` – Shows the last 10 lines of a file.

- Example: `tail -f logfile.log` (live logs).

File Permissions and Ownership

15. `ls -l` – Displays file permissions and ownership details.

16. `chmod` – Changes file permissions (read, write, execute).

- Example: `chmod 755 script.sh`

17. `chown` – Changes file ownership.

- Example: `chown user:group file.txt`

18. `umask` – Sets default permissions for new files.

Process Management Commands

19. `ps` – Displays currently running processes.

- Example: `ps aux`

20. top – Real-time view of running processes and system usage.
21. htop – Advanced version of top (interactive).
22. kill – Terminates a process by PID.
 - Example: kill -9 1234
23. jobs – Lists background jobs started in the shell.
24. fg – Brings a background job to the foreground.
25. bg – Resumes a suspended job in the background.
26. nice – Starts a process with a specific priority.
27. renice – Changes the priority of a running process.

Windows Programming

- **Windows OS** (by Microsoft) provides a **GUI-based multitasking environment**.
- **Windows Programming** refers to writing applications that run on Windows using its **APIs and frameworks**.

Windows Programming Models

1. Win32 API

- Low-level C-based API for GUI and system programming.
- Example: window creation, event handling.

2. MFC (Microsoft Foundation Classes)

- Object-oriented C++ wrapper over Win32.

3. .NET Framework / C#

- High-level programming with managed code.
- Rich GUI, database, and network libraries.

4. UWP (Universal Windows Platform)

- For apps that run across PC, tablet, mobile, Xbox.

Basic Concepts of Windows Programming

- **Event-driven programming** → Windows apps respond to events (mouse click, key press).
- **Message Loop** → Core mechanism for event handling.
- **Window Procedure** → Function that processes messages for a window.

Windows Application Structure

1. **Include header files** → <windows.h>.
2. **WinMain()** → Entry point of Windows program.
3. **Message Loop** → Processes messages from OS.
4. **WndProc()** → Handles events (paint, resize, close).

GUI Components in Windows

- **Controls**: Buttons, Textboxes, Menus.
- **GDI (Graphics Device Interface)**: Drawing graphics, fonts, images.
- **Dialogs**: Common dialogs (Open File, Save File).
- **Events**: Mouse click, key press, resize, timer.

Advanced Windows Programming

- **Multithreading:** CreateThread() API.
- **File I/O:** CreateFile(), ReadFile(), WriteFile().
- **Networking:** Winsock API.
- **Registry Access:** RegOpenKeyEx().
- **COM (Component Object Model):** Software component architecture.

Modern Windows Programming

- **.NET & C#** → Easier GUI development with Windows Forms & WPF.
- **UWP** → Cross-device apps with XAML and C#.
- **WinUI 3** → Latest Windows GUI framework.

Applications of Windows Programming

- Desktop applications (Notepad, Paint).
- Multimedia apps.
- Device drivers (via Win32 APIs).
- Business and database software.